AUTONOMOUS AGENTS · SERIES TWO

The Scholar Series

Lesson 4 — Progress Memory.
Over a long job, the agent must remember not just facts, but where it stands.

Still building in LangGraph and CrewAI side by side

Where we are. Our agent loops (Lesson 1), plans its own work (Lesson 2), and critiques and revises its drafts (Lesson 3). Each piece it produces is now good. But across a long assignment of many steps, it loses the thread — half-forgetting what it has covered and revisiting finished sections. This lesson gives it *progress memory*: a structured, working record of the state of the whole job, which is a different kind of memory than series one taught.

What this lesson covers

1. **The scene** — the agent losing track halfway through a long report
2. **The concept** — working memory of progress, not memory of facts
3. **The architecture** — a structured progress record the loop reads and updates
4. **Build it in LangGraph** — progress as first-class state
5. **Build it in CrewAI** — built-in memory and task outputs
6. **Compare** — explicit state vs framework-managed memory
7. **Run it + the new crack** — and the autonomy that still needs a human

PART 1

The scene: lost in the middle of a long job

Give our Lesson 3 agent a small task and it shines. But ask it to produce a long, multi-section report — say a fifteen-section market analysis — and somewhere in the middle, things start to go wrong in a very particular way.

Goal: Write a 15-section analysis of the solid-state battery market.

```
section 6 → written and approved
section 11 → starts re-researching section 6's topic, not realising it's done
section 13 → contradicts a figure it stated back in section 4
section 14 → asks itself "have I covered manufacturing yet?" — it had, in section 9
```

The agent is not failing at any individual section; each one, in isolation, is well-researched and well-revised. It is failing at *keeping track of the whole*. By section eleven, the running pile of everything it has done — the plan, dozens of findings, several drafts, many critiques — has grown so large and unstructured that the agent can no longer reliably answer simple questions about its own progress: what have I finished, what am I working on, what remains, and what did I already decide? It is like a writer with a desk so buried in papers that they redo work they have already completed.

THE FAILURE, PRECISELY

The agent has no organised, queryable record of its own progress through a long task. Everything it has done is jumbled together in an ever-growing context, so it can't reliably tell what is done versus pending, and it repeats or contradicts itself. What's missing is a structured *working memory* of the state of the job itself.

PART 2

The concept: remembering where you are, not just what you know

Here is a distinction worth pausing on, because it's easy to conflate two things that the word "memory" lumps together. Series one taught you one kind of memory: remembering *facts*, like a customer's name or their past orders. That is memory *about the world*. What our autonomous agent needs now is a

different kind entirely: memory of *its own progress through a task*. That is memory *about itself* — a working record of what it has done, what it is doing, and what it still must do.

This second kind is often called working memory or scratchpad memory, and the key to making it useful is that it must be *structured*, not just a growing blob of text. Instead of piling every finding and draft into one undifferentiated heap, the agent maintains an organised record: a checklist of sections with their status (done, in-draft, pending), the key decisions made so far, and the facts already established. Crucially, this record stays compact and queryable even as the work grows, so the agent can always answer "where am I?" by reading a tidy summary rather than re-reading everything it has ever done.

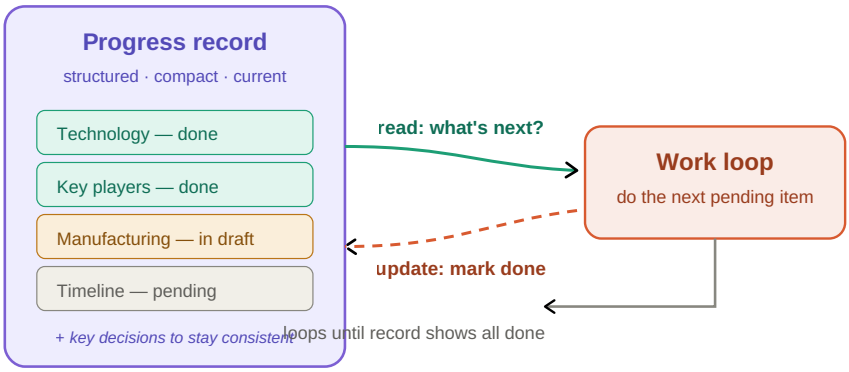
Think of the difference between a researcher's overflowing pile of source papers and the neat project tracker pinned to their wall. The pile is everything they know; the tracker is where they *are* — which chapters are drafted, which are in review, which are untouched, and the few decisions that must stay consistent throughout. The tracker is small, structured, and always current, and it's what lets the researcher pick up a long project after a break and instantly know what to do next. Progress memory is that tracker, for the agent.

> **THE ONE IDEA OF THIS LESSON**
>
> Progress memory is a structured, compact, continuously-updated record of the agent's own state in a long task — what is done, in progress, and pending, plus key decisions to stay consistent with. It differs from series one's fact-memory: this is memory *about the work itself*, and it's what lets an agent sustain a long job without losing the thread.

PART 3

The architecture: a progress record beside the loop

The architecture adds a structured progress record that sits alongside the working loop. Every pass through the loop begins by *reading* the record to decide what to do next, and ends by *updating* it to reflect what was just accomplished. The record is the agent's single source of truth about its own state.



The progress record sits beside the loop. Each iteration reads it to choose the next action and writes back to it on completion. Because the record is structured and compact, the agent always knows exactly where it stands.

The vital detail is the two arrows between the loop and the record: a read at the start of each step and a write at the end. This read-then-act-then-update rhythm is what keeps the agent oriented. Without the read, it wouldn't know what's next; without the write, the record would go stale. Together they give the agent a reliable, always-current sense of its own progress, no matter how long the job runs.

PART 4

Build it in LangGraph: progress as first-class state

This is where LangGraph's design philosophy pays its biggest dividend in the whole series. Recall that from Lesson 1, LangGraph has always carried an explicit shared *state* object. Progress memory is simply a richer, more structured version of that state — so for LangGraph, this lesson is less about adding a new mechanism and more about using the state we already have more deliberately. The progress record *is* the state.

```
from typing import TypedDict, List, Dict
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class ProgressState(TypedDict):
    goal: str
    # THE PROGRESS RECORD: a structured tracker, not a blob of text.
```

```
    sections: List[Dict]      # each: {"title":..., "status":..., "content":...}
    decisions: List[str]      # key facts/choices to stay consistent with
```

Why this exists: `sections` *with explicit statuses is the structured tracker. It replaces the jumbled pile with a record the agent can query: which sections are 'done', which are 'pending'.*

```
def next_step(state: ProgressState) -> ProgressState:
    # READ the record: find the first section still pending.
    pending = [s for s in state["sections"] if s["status"] == "pending"]
    if not pending:
        return state
    section = pending[0]

    # ACT: write this section, giving the model the decisions made so far
    # so it stays consistent and doesn't contradict earlier sections.
    section["content"] = llm.invoke([
        SystemMessage(content=f"Write section '{section['title']}'. Stay "
                              f"consistent with these decisions:
{state['decisions']}"),
        HumanMessage(content=state["goal"])]).content

    # UPDATE the record: mark done so we never redo it.
    section["status"] = "done"
    return state

def work_remaining(state: ProgressState) -> str:
    # The record itself is the exit condition: loop until nothing is pending.
    return "continue" if any(s["status"] == "pending" for s in state["sections"])
else "stop"
```

Why this exists: notice the read-act-update rhythm. `next_step` *reads the record to find what's pending, acts, then updates the status. The agent can never lose track because the record always reflects reality.*

```
graph = StateGraph(ProgressState)
graph.add_node("step", next_step)
graph.set_entry_point("step")
graph.add_conditional_edges("step", work_remaining,
                            {"continue": "step", "stop": END})
agent = graph.compile()
```

Why this exists: the loop is small because the intelligence lives in the structured state. The agent's progress is no longer an emergent property of a giant context — it's an explicit record you could print at any moment.

PART 5

Build it in CrewAI: built-in memory and task outputs

CrewAI takes a strikingly different approach, and it reveals the framework's whole philosophy. Rather than asking you to hand-design a structured state object, CrewAI offers *built-in memory* that you switch on, and it tracks task outputs for you. You enable memory with a flag, and the framework retains what agents have produced and learned across the run, making it available to later steps automatically.

```
from crewai import Agent, Task, Crew, Process

analyst = Agent(
    role="Market Analyst",
    goal="Write each section of the report, consistent with prior sections",
    backstory="You track what you've written and never contradict yourself.",
    memory=True,    # this agent remembers across the run
)

# Build one task per section. CrewAI runs them in order and, with
# memory on, each task can draw on what earlier tasks established.
sections = ["Technology", "Key players", "Manufacturing", "Timeline"]
tasks = [
    Task(description=f"Write the '{s}' section, consistent with earlier ones.",
         expected_output=f"The '{s}' section.", agent=analyst)
    for s in sections
]
```

Why this exists: the `memory=True` flag is the whole mechanism. Where LangGraph asks you to design the progress record, CrewAI provides a managed memory and asks you to trust it to carry context between tasks.

```
# Turn on crew-level memory too: CrewAI keeps short- and long-term
# memory of the run, so later sections "remember" earlier ones.
crew = Crew(
    agents=[analyst], tasks=tasks,
    process=Process.sequential,
    memory=True,    # the framework manages the progress context for you
)
result = crew.kickoff()
```

Why this exists: `memory=True` at the crew level hands the whole problem to CrewAI. It's dramatically less code than LangGraph's explicit record — but you no longer hold the progress as an object you can inspect.

PART 6

Compare: explicit state vs managed memory

This comparison is the sharpest expression of the two philosophies in the entire series, because progress memory is exactly the kind of thing where "build it yourself" and "let the framework handle it" diverge most.

LANGGRAPH VS CREWAI — PROGRESS MEMORY

	LangGraph	CrewAI
The record	You design it as explicit, structured state	Built-in memory you switch on with a flag
Inspectable	Fully — print or query the record anytime	Largely managed by the framework, less visible
Control	You decide exactly what's tracked and how	The framework decides what to retain
Effort	More — you build the tracker	One flag — the least code by far
Best when	Progress shape matters and must be auditable	You want sensible memory without designing it

The honest trade-off here is about *ownership of the truth*. With LangGraph, the progress record is an object you designed and can examine at any moment — invaluable when the structure of progress matters, when you must debug why the agent did something, or when correctness across a long job is critical. With CrewAI, you trade that visibility for extraordinary convenience: a single `memory=True` flag gives you working memory without designing a thing, but the framework decides what to keep and you have less insight into it. For a quick autonomous agent, CrewAI's managed memory is a gift. For a system where you must guarantee and audit consistency across many steps, LangGraph's explicit record is worth the extra code. This is the clearest case in the series of matching the tool to how much you need to *see*.

PART 7

Run it, and the new crack

Run either version on the long report and the losing-the-thread problem vanishes. The agent marches through its sections without repeating or contradicting itself, because it always knows where it stands.

Goal: Write the multi-section market analysis.

```
reads record → "Manufacturing" is pending → writes it → marks done
reads record → all sections done, decisions consistent → stops
```

Scholar: A complete, internally consistent report — every section written once, no contradictions, because the progress record kept the whole job in view.

Scholar is now genuinely formidable. It runs autonomously, plans its own work, critiques and revises for quality, and sustains a long, complex job without losing the thread. It can take a single goal and produce a real, polished report entirely on its own. And that very capability is precisely what raises the final, most important question of this whole series. Scholar now does a great deal of consequential work

with no human watching. What if its self-authored plan misses something crucial? What if it confidently commits to a wrong decision in section two that then propagates through the whole report? What if the topic is sensitive and a human really ought to sign off before the work goes out? An agent this autonomous needs a way to *pause and check in* — not despite its autonomy, but precisely because of it.

THE PROBLEM LESSON 5 WILL FIX

Scholar can now do extensive, consequential work entirely unsupervised — which is powerful and, unchecked, risky. In Lesson 5, the capstone, we'll add the *human-in-the-loop checkpoint*: the ability to pause the agent at key moments for a human to review, approve, or redirect before it proceeds. This is the responsible counterweight to autonomy, and it's the natural place for this series to end.

End of Scholar Lesson 4. You gave the agent a structured memory of its own progress, in two frameworks, and saw the starkest version yet of "build it yourself" versus "let the framework handle it." Next, the finale: keeping a human in the loop.